Question No. 01 (Data Structures - 04 Pts): Generally, we say that linked lists offer more efficient insertion and deletion as compared to arrays. However, in fact, for both data structures, both insertion and deletion take linear time. That is time complexity of insertion/deletion in array and linked list is O(n) where n is number of elements. Briefly explain why insertion and deletion take linear time in both arrays and linked lists.

Solution:

Question No. 02 (Arrays - 06 Pts): Let's assume the following array, in which values are sorted in ascending order. Suppose we performed a **binary search** on this array, searching for the number 40.

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Value	10	15	20	25	30	35	40	45	50	55	60

Your task is to show low index (low), high index (high), mid and the value compared at each step as the search proceeds.

	Initially	**Step 1**	**Step 2**	**Step 3**	**Step 4**	**Step 5**	**Step 6**
Low							
High							
Mid							
Value							

Question No. 03 (Arrays - 05 Pts): A square matrix (that is n x n matrix) is called a lower-triangular matrix if all the values above its diagonal are zeros (0s). For example, in the following M1 is a lower-triangular matrix but M2 is not.

M1 =

5	0	0	0
7	6	0	0
3	9	2	0
4	4	1	7

M2=

5	0	7	0
7	6	0	8
3	9	2	0
4	4	1	7

Write a method to determine if a given matrix is a lower-triangular matrix or not.

Solution:

Java

```
public boolean Matrix(int M[][]) {









}
```

Question No. 04 (Linked List - 10 Pts): Write a recursive method that operates on an existing linked list of integers. Your method should print the nodes in reverse order as well as the sum of all values in the linked list. Assume that hptr initially points to head of the link list and sum is zero.

For example:

Linked List: 51 -> 4 -> 2 -> 9 -> 12

Display: 12, 9, 2, 4, 51

Sum Returned: 78

Solution:

```
public int display(LL_node hptr) {




}
```

Question No. 05 (Recursion - 10 Pts): Trace the following method by calling it with exam05(3627481). You must draw ALL recursive calls (draw the boxes!) and you must show the final answer that is returned. No credit for the final answer only.

```
public int exam05(int number) {
  if (number == 0) {
    return 0;
  }
  else {
    if ((number % 10) % 2 != 0) {
      return number % 10 + exam05(number/10);
    }
    else {
      return exam05(number/10);
    }
  }
}
```

Solution:

Question No. 06 (Algorithm Analysis - 05 Pts): Algorithm A runs in $O(n^3)$ time. For an input of size 4, the running time is 192 milliseconds. How long will Algorithm A take to run on an input of size 8?

Solution:

Step 1: Find c (2.5 pts)	**Step 2: Find T(n) (2.5 pts)**

Question No. 07 (Algorithm Analysis - 05 Pts): Determine the Big-O running time of the following code fragment, in terms of the variable n. **YOU MUST EXPLAIN YOUR ANSWER**

Solution:

Code	**Big-O and Explanation**

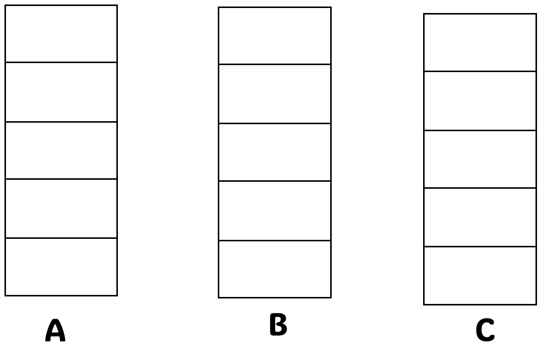
for (int i = 0; i < n; i++) {
  j=n;
  while (j < 1) {
  j = j / 2;
  d = d + 2;
  }
}	

Question No. 08 (Stacks - 15 Pts): Convert the following infix expression into its equivalent postfix expression using a stack. Additionally, you must show the contents of the stack at the indicated points (A, B, and C) in the infix expression.

A B C

2 + 10 * ((21 + 3)   / 4) - 14 /   (30 + 26)

Solution:



Postfix expression: ______________________________ (3points)